Digital Electronics & Systems

19ECE204/19CSE203

LECTURE 5

XOR and XNOR Logic Gates

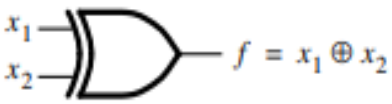
Exclusive-OR (XOR) Gate

- So far we have encountered AND, OR, and NOT gates as the basic elements from which logic circuits can be constructed.
- There is another basic element that is very useful in practice, particularly for building circuits that perform arithmetic operations.
- This element realizes the Exclusive-OR function.

For n-input XOR gate output is high, when odd number of inputs are high

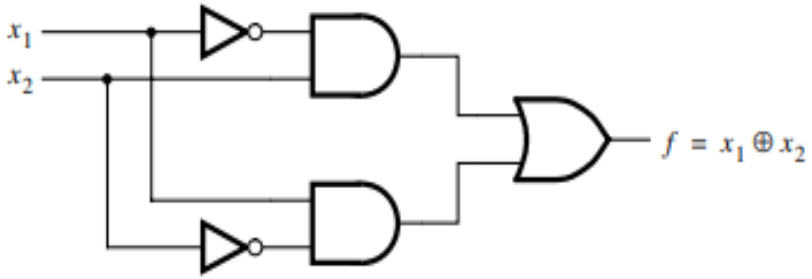
x_1	x_2	$f = x_1 \oplus x_2$
0	0	0
0	1	1
1	0	1
1	1	0

(a) Truth table



(b) Graphical symbol

XOR and XNOR Logic Gates



(c) Sum-of-products implementation

- The truth table for this function is similar to the OR function except that f = 0 when both inputs are 1.
- Because of this similarity, the function is called Exclusive-OR, which is commonly abbreviated as XOR.
- The XOR operation is usually denoted with the $\oplus$ symbol. It can be realized in the sum-of-products form as

$$x1 \oplus x2 = \overline{x1}x2 + x1\overline{x2}$$

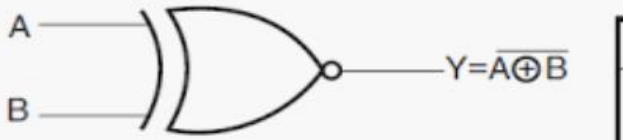
XOR and XNOR Logic Gates

Exclusive-NOR (XNOR) Gate

- The Exclusive-NOR Gate function is a digital logic gate that is the reverse or complementary form of the Exclusive-OR function.
- Basically the "Exclusive-NOR" gate is a combination of the Exclusive-OR gate and the NOT gate.

2-input "Ex-OR" gate plus a "NOT" gate

The logical XNOR operation is represented by ⊙. That is a dot surrounded by circle.



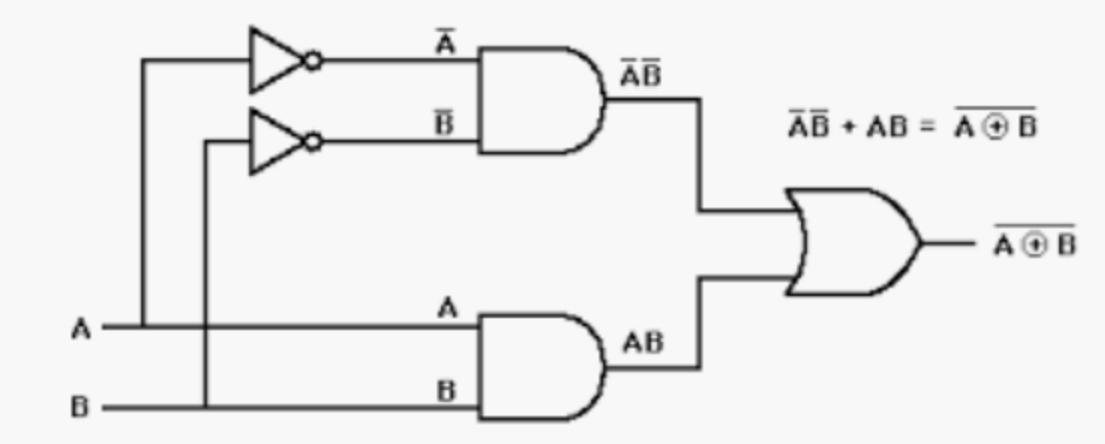
$$Y = (\overline{A \oplus B}) = (A.B + \overline{A}.\overline{B}) = A \odot B$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

XOR and XNOR Logic Gates

Exclusive-NOR (XNOR) Gate

- The output of a 2-input XNOR gate is high when all of its inputs are same, either high or low. If its inputs are different then the output of the XNOR gate is low.
- For n-input XNOR gate, output is high when even number of inputs are high.

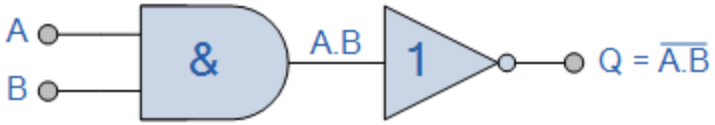


SOP Implementation

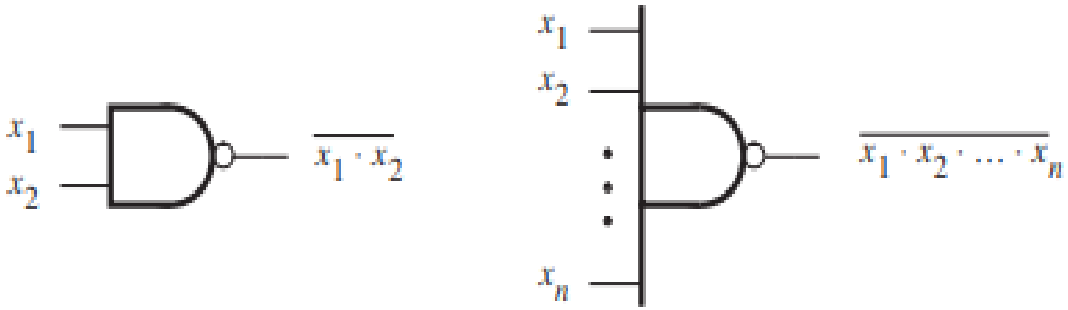
NAND and NOR Logic Networks

- The NAND and NOR gates are called Universal gates since with either all other logic gates can be implemented.
- **NAND Gate-**

➢ NAND gate is equal to AND gate followed by NOT gate.



Logic NAND Gate Equivalence



Symbol for NAND Gate

NAND and NOR Logic Networks

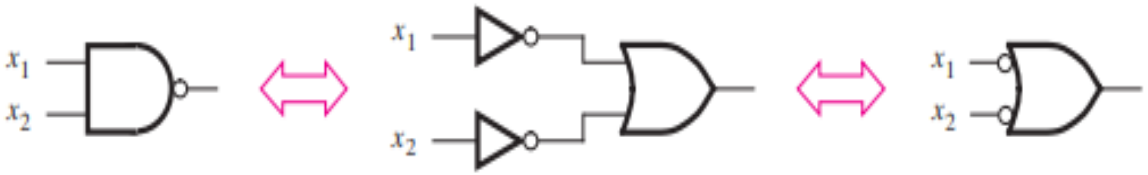
NAND Gate-

Truth Table

2 Input NAND gate		
A	B	$\overline{A.B}$
0	0	1
0	1	1
1	0	1
1	1	0

In NAND gate, if any input is LOW (0), a HIGH (1) output results.

- A 2-input NAND gate is equivalent to 2-input OR gate with complemented inputs.

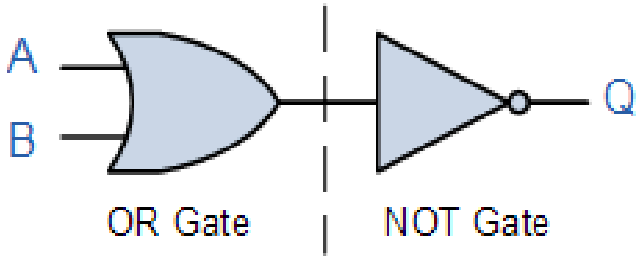


$$\overline{x_1 x_2} = \bar{x}_1 + \bar{x}_2$$

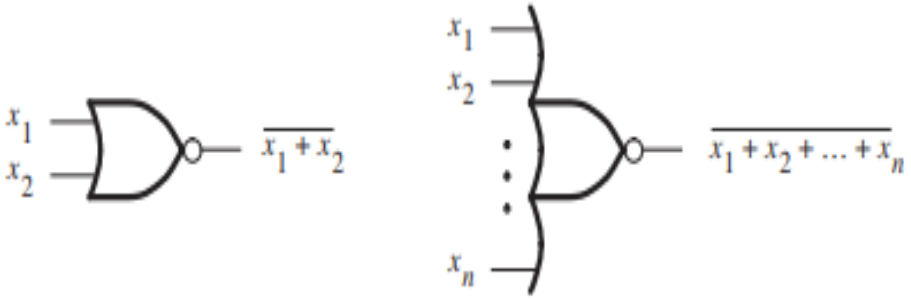
NAND and NOR Logic Network

NOR Gate-

- NOR gate is equal to OR gate followed by NOT gate.



Logic NOR Gate Equivalence



Symbol for NOR Gate

NAND and NOR Logic Networks

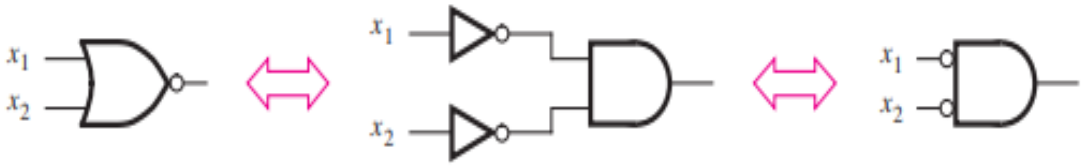
NOR Gate-

Truth Table

2 Input NOR gate		
A	B	$\overline{A+B}$
0	0	1
0	1	0
1	0	0
1	1	0

The output of **NOR gate** is high only if all the inputs are low.

- A 2-input NOR gate is equivalent to 2-input AND gate with complemented inputs.



(b) $\overline{x_1 + x_2} = \bar{x}_1 \bar{x}_2$

NAND and NOR Logic Networks

Function Implementation Using NAND Gates Only

- Sum-of-Product form (SOP Form)-

➢ First Method:

1) Draw AND-OR Network for the given function.

2) Each connection between the AND gate and an OR gate is replaced by a connection that includes two inversions of the signal: one inversion at the output of the AND gate and the other at the input of the OR gate.

3) Such double inversion has no effect on the behavior of the network.

Consider a function

$$f = x_1.x_2 + x_3.x_4.x_5$$

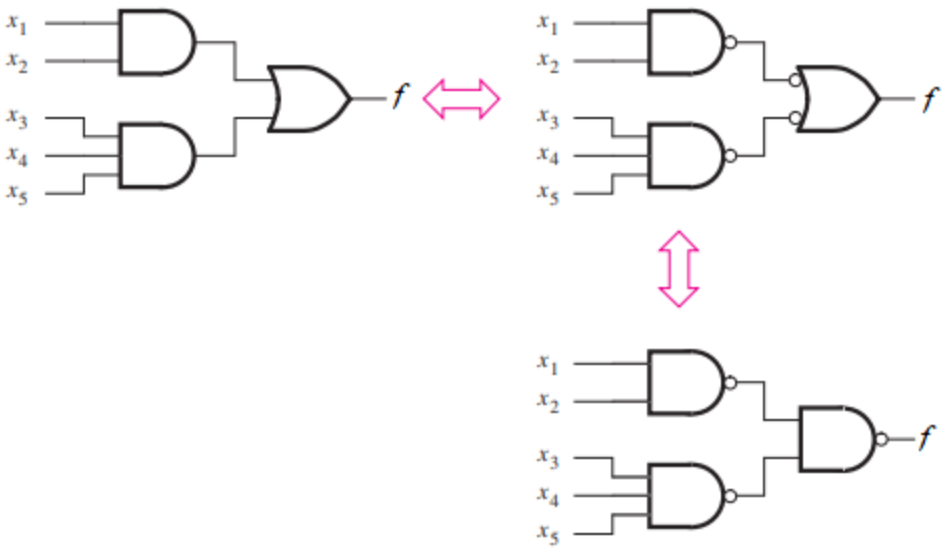
NAND and NOR Logic Networks

Function Implementation Using NAND Gates Only

- Sum-of-Product form (SOP Form)-

➢ First Method:

$$f = x_1.x_2 + x_3.x_4.x_5$$



any AND-OR network
can be implemented as
a NAND-NAND network
having the same topology

Using NAND gates to implement a sum-of-products

NAND and NOR Logic Networks

Function Implementation Using NAND Gates Only

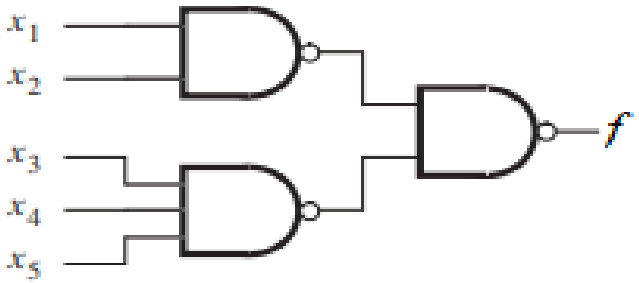
- Sum-of-Product form (SOP Form)-

➢ Second Method:

$$f = x_1.x_2 + x_3.x_4.x_5$$

$$f = \overline{\overline{x_1.x_2 + x_3.x_4.x_5}}$$

$$f = \overline{(\overline{x_1.x_2}).\ (\overline{x_3.x_4.x_5})}$$ (Using Demorgan's Theorem)



Using NAND gates to implement a sum-of-products

NAND and NOR Logic Networks

Function Implementation Using NOR Gates Only

- Product-of-Sum form (POS Form)-

➢ First Method:

1) Draw OR-AND Network for the given function.

2) Each connection between the OR gate and AND gate is replaced by a connection that includes two inversions of the signal: one inversion at the output of the OR gate and the other at the input of the AND gate.

3) Such double inversion has no effect on the behavior of the network.

Consider a function

$$f = (x_1+x_2) \,.\, (x_3+x_4+x_5)$$

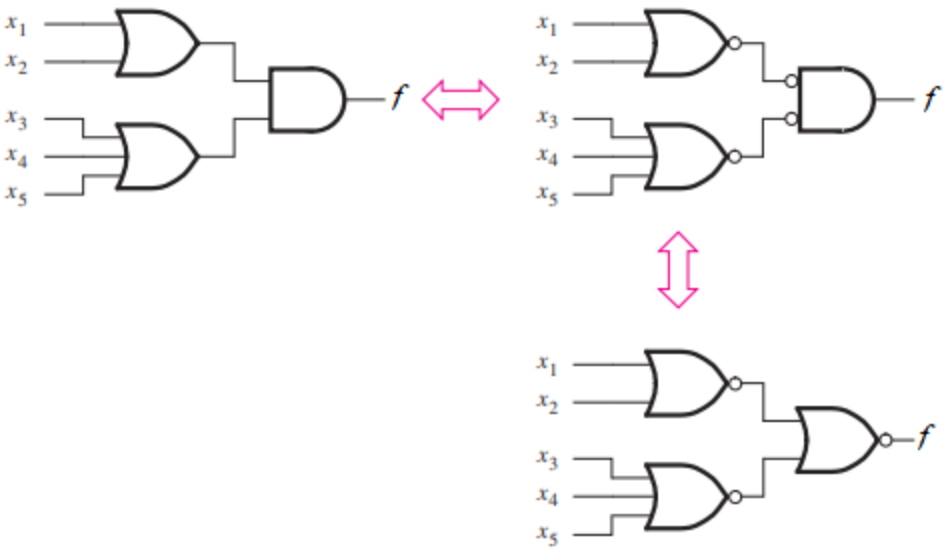
NAND and NOR Logic Networks

Function Implementation Using NOR Gates Only

- Product-of-Sum form (POS Form)-

➢ First Method:

$$f = (x_1+x_2) \,.\, (x_3+x_4+x_5)$$



any OR-AND network can be implemented as a NOR-NOR network having the same topology

Using NOR gates to implement a product-of-sums

NAND and NOR Logic Networks

Function Implementation Using NOR Gates Only

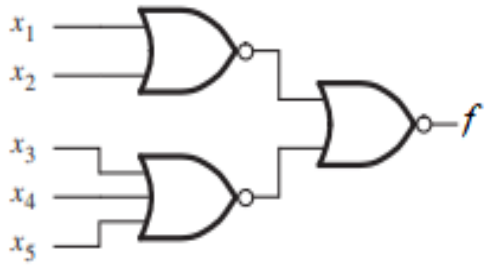
- Product-of-Sum form (POS Form)-

➢ Second Method:

$$f = (x_1+x_2) \cdot (x_3+x_4+x_5)$$

$$f = \overline{(x_1+x_2) \cdot (x_3+x_4+x_5)}$$

$$f = \overline{\overline{(x_1+x_2)}+\overline{(x_3+x_4+x_5)}}$$ (Using Demorgan's Theorem)

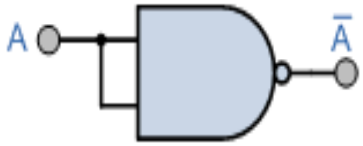


Using NOR gates to implement a product-of-sums

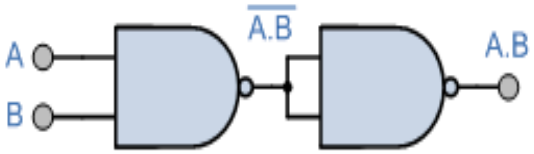
NAND and NOR Logic Networks

Various Logic Gates Using Only NAND Gates-

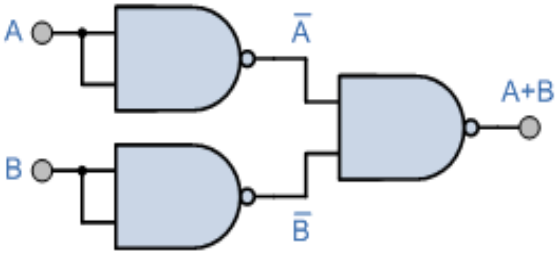
1) NOT Gate



2) AND Gate



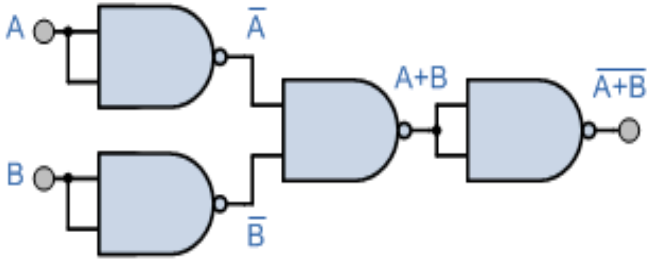
3) OR Gate



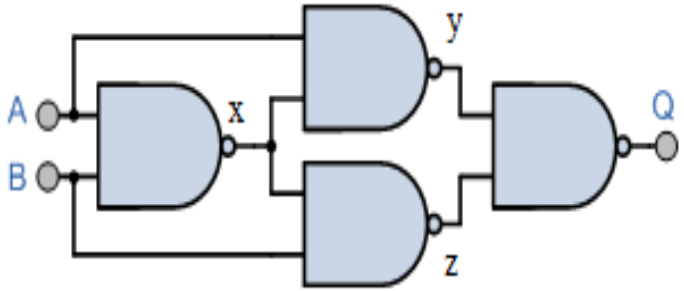
NAND and NOR Logic Networks

Various Logic Gates Using Only NAND Gates-

4) NOR Gate



5) XOR Gate



$$x = \overline{A.B} = \overline{A} + \overline{B}$$

$$y = \overline{A.x} = A.(\overline{A} + \overline{B})$$

$$y = A.\overline{B}$$

$$z = \overline{B.X} = B.(\overline{A} + \overline{B})$$

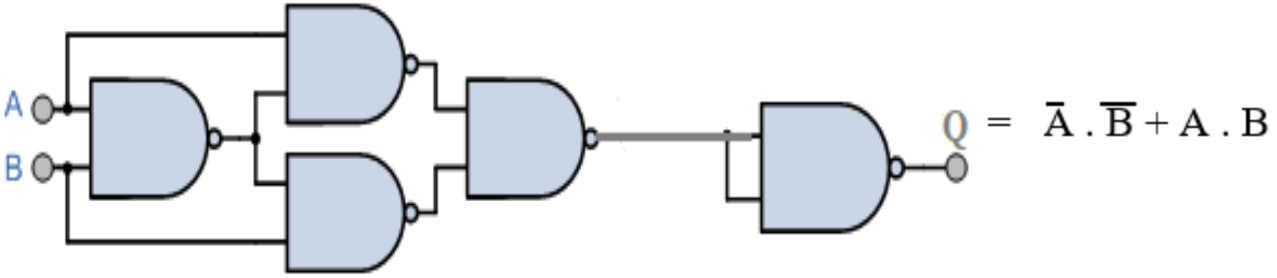
$$z = \overline{A}.B$$

$$Q = \overline{y.z} = A.\overline{B} + \overline{A}.B$$

NAND and NOR Logic Networks

Various Logic Gates Using Only NAND Gates-

6) XNOR Gate

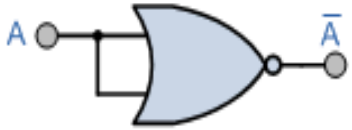


LECTURE 6

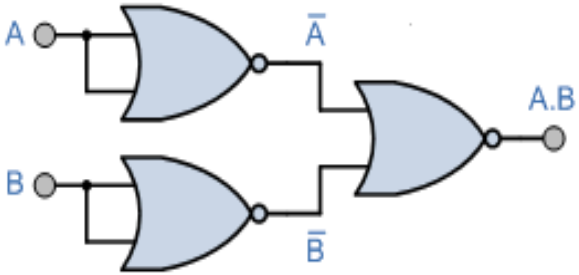
NAND and NOR Logic Networks

Various Logic Gates Using Only NOR Gates-

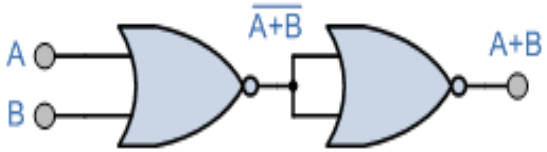
1) NOT Gate



2) AND Gate



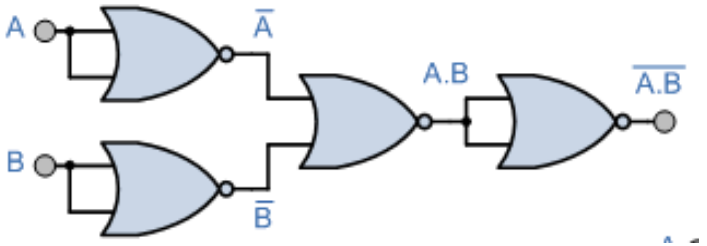
3) OR Gate



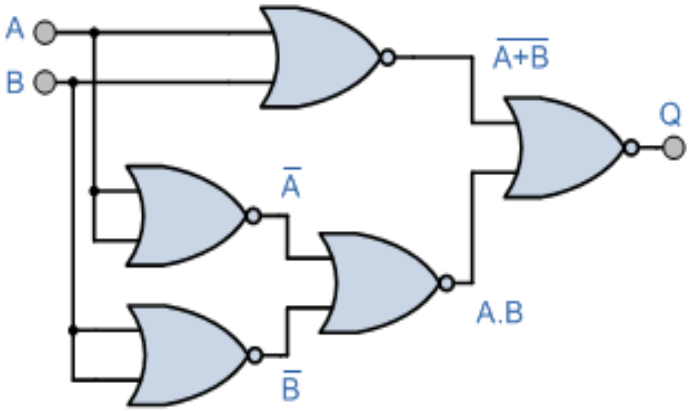
NAND and NOR Logic Networks

Various Logic Gates Using Only NOR Gates-

4) NAND Gate



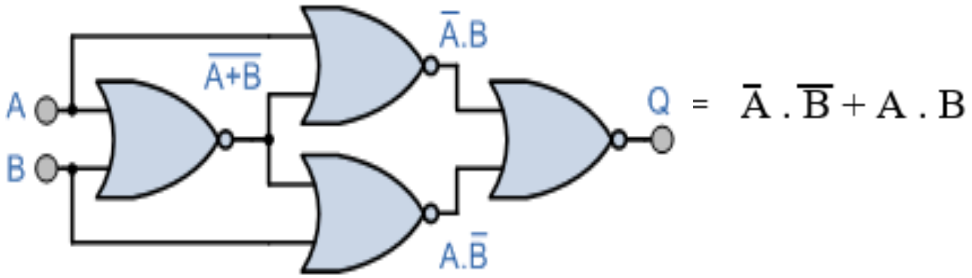
5) XOR Gate



NAND and NOR Logic Networks

Various Logic Gates Using Only NOR Gates-

6) XNOR Gate



NAND and NOR Logic Networks

Function Implementation Using NAND Gates Only

Example 1-

Implement the function f (a, b, c) = Σ m (2,3,4,6,7) using NAND gates only.

(a) In SOP form (b) In POS form

Solution- The canonical SOP expression for the function is derived using minterms-

$$f = m_2 + m_3 + m_4 + m_6 + m_7$$

$$f = a'bc' + a'bc + ab'c' + abc' + abc$$

$$f = a'b\,(c'+ c) + ac'\,(b'+ b) + ab\,(c'+c) \qquad \{x + x' = 1\}$$

$$f = a'b + ac' + ab = (a'+a)\,b + ac'$$

$$f = b + ac'$$

NAND and NOR Logic Networks

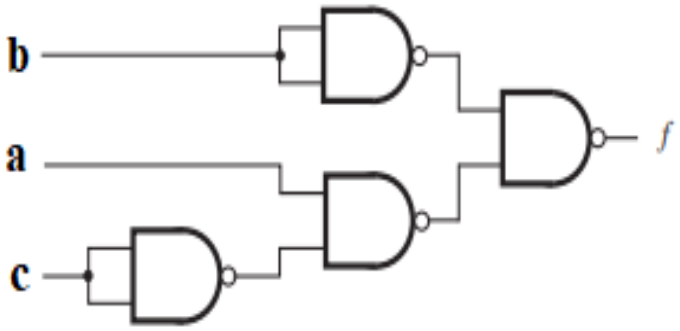
Function Implementation Using NAND Gates Only

Example 1 Solution-

(a) SOP form

$f = b + ac' = \overline{\overline{b + ac'}}$ $\quad x' = \overline{x} = \text{complement of } x\}$

$f = \overline{b' \,.\, (ac')'}$



Function implementation using NAND Gates only

NAND and NOR Logic Networks

Function Implementation Using NAND Gates Only

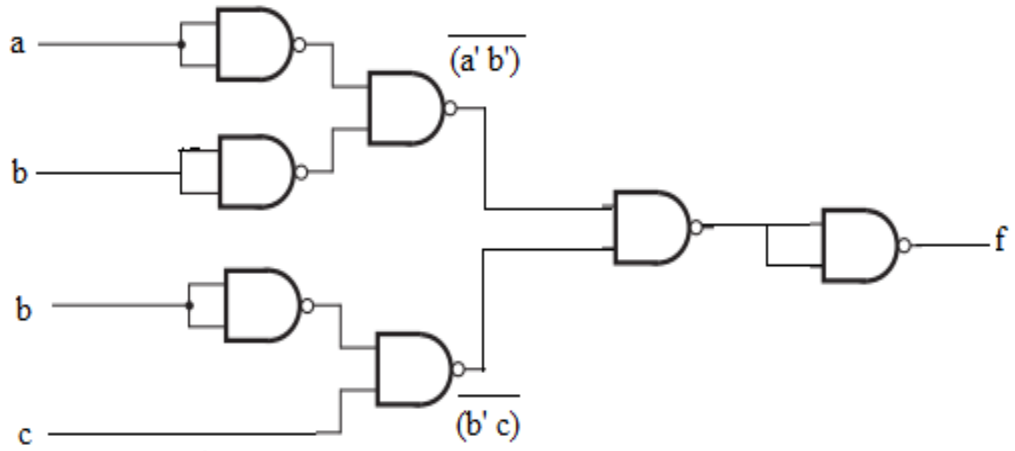
Example 1 Solution-

(b) POS form

$f = b + ac' = \overline{\overline{(a + b)}}. (b + c')$ {Distributive Law}

$f = \overline{\overline{(a + b) . (b + c')}}$ {$x' = \overline{x}$ = complement of x}

$f = \overline{(a'. b') + (b'. c)} = \overline{(a'. b')} . \overline{(b'. c)}$ {Demorgan's Theorem}



Function implementation using NAND Gates only

NAND and NOR Logic Networks

Function Implementation Using NAND Gates Only

Example 1 Solution-

$f(a, b, c) = \Sigma m(2,3,4,6,7) = b + ac' = (a + b).(b + c')$

- Given function implementation in SOP form- No. of NAND gates required – 4
- Given function implementation in POS form- No. of NAND gates required - 7

Conclusion- Any logic function can be implemented using NAND gates. To achieve this, first the logic function has to be written in Sum of Product (SOP) form. Once logic function is converted to SOP, then is very easy to implement using NAND gate and it requires less number of gates.

NAND and NOR Logic Networks

Function Implementation Using NOR Gates Only

Example 2-

Implement the function f (a, b, c) = Σ m (2,3,4,6,7) using NOR gates only.

(a) In SOP form (b) In POS form

Solution- From Example 1 we got-

$$f = b + a\,c'$$

(a) In SOP form

$$f = \overline{\overline{b + a\,c'}} = \overline{b' \,.\, \overline{a\,c'}} \qquad \{\text{Demorgan's Theorem}\}$$

$$f = b' \,.\, \overline{(a' + c)}$$

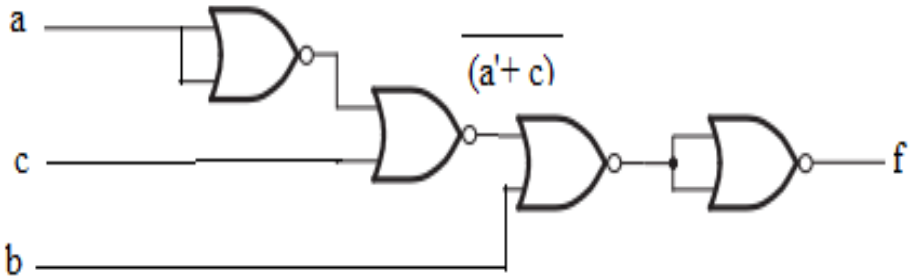
$$f = b + (a' + c)$$

NAND and NOR Logic Networks

Function Implementation Using NOR Gates Only

Example 2 Solution-

(a) In SOP form



Function implementation using NOR Gates only

NAND and NOR Logic Networks

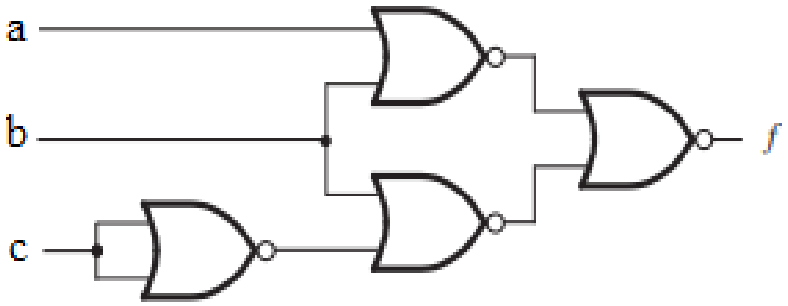
Function Implementation Using NOR Gates Only

Example 2 Solution-

(a) In POS form

$$f = b + a\,c' = \overline{\overline{(a + b)\,.\,(b + c')}}$$

$$f = \overline{\overline{(a + b)} + \overline{(b + c')}}$$



Function implementation using NOR Gates only

NAND and NOR Logic Networks

AMRITA VISHWA VIDYAPEETHAM

Function Implementation Using NOR Gates Only

Example 2 Solution-

Conclusion- Any logic function can be implemented using NOR gates. To achieve this, first the logic function has to be written in Product of Sum (POS) form. Once logic function is converted to POS, then is very easy to implement using NOR gate and it requires less number of gates.

(in example 2 number of NOR gates, in both SOP and POS form implementations, are same but it is not applicable on all the functions)

NAND and NOR Logic Networks

Example 3-

Design the simplest circuit that implements the given function using NAND gates only.

$F(x, y, z) = \Sigma m(1,2,3,4,5,7)$

Solution- The canonical SOP expression for the function is derived using minterms-

$F = m_1 + m_2 + m_3 + m_4 + m_5 + m_7$

$F = x'y'z + x'yz' + x'yz + xy'z' + xy'z + xyz$

$F = x'y'z + x'yz + x'yz' + x'yz + xy'z' + xy'z + xy'z + xyz$

$F = x'z(y' + y) + x'y(z' + z) + xy'(z' + z) + xz(y' + y)$ $\{x + x' = 1\}$

$F = x'z + x'y + xy' + xz = (x' + x)z + x'y + xy'$

$F = xy' + x'y + z$

NAND and NOR Logic Networks

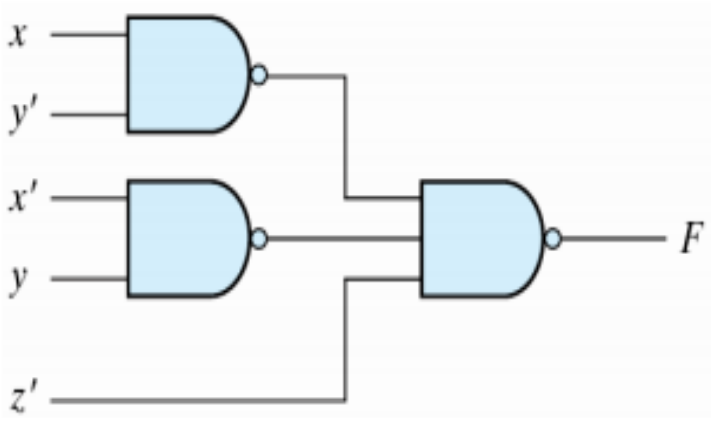
Example 3 Solution-

F = xy' +x'y + z

$$F(x,y,z) = xy' + x'y + z$$

$$= \overline{\overline{xy' + x'y + z}}$$

$$= \overline{\left[\overline{(xy')}\ \overline{(x'y)}\ z'\right]}$$



(Assuming both normal and complemented inputs are available)

NAND and NOR Logic Networks

Example 4-

Design the simplest circuit that implements the given function using NOR gates only.

$$F(x, y, z) = \Sigma m(1,2,3,4,5,7)$$

Solution-

$$F(x, y, z) = \Sigma m(1,2,3,4,5,7) = \Pi M(0,6)$$

The canonical POS expression for the function is derived using maxterms-

$$F = M_0 \cdot M_6$$

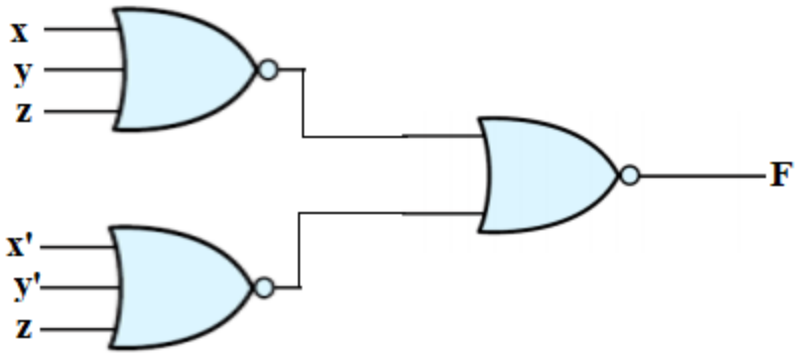
$$F = (x + y + z) \cdot (x' + y' + z) = \overline{\overline{(x + y + z) \cdot (x' + y' + z)}}$$

$$F = \overline{\overline{(x + y + z)} + \overline{(x' + y' + z)}}$$

NAND and NOR Logic Networks

Example 4 Solution-

$$F = \overline{\overline{(x + y + z)} + \overline{(x' + y' + z)}}$$



LECTURE 7

Optimized Implementation of Logic Functions

- The algebraic manipulation can be used to find the lowest-cost implementations of logic functions.
- It is easy to derive a straightforward realization of a logic function in a canonical form, but it is not at all obvious how to choose and apply the theorems and properties of Boolean algebra to find a minimum-cost circuit.
- Indeed, the algebraic manipulation is rather tedious and quite impractical for functions of many variables.
- Another minimization approach, which provides a neat way to manually derive minimum-cost implementations of logic functions, is known as Karnaugh Map.

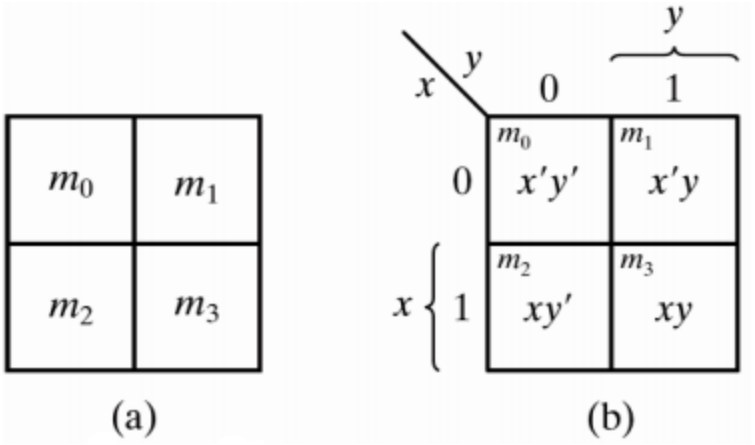
Optimized Implementation of Logic Functions – Karnaugh Map

- The Karnaugh map approach provides a systematic way of performing this optimization.
- The Karnaugh map (K-Map) is an alternative to the truth-table form for representing a function. It is A pictorial form of a truth table.
- The K-Map is a diagram made up of squares. Each square is a cell of K-map. The map consists of cells that correspond to the rows of the truth table.

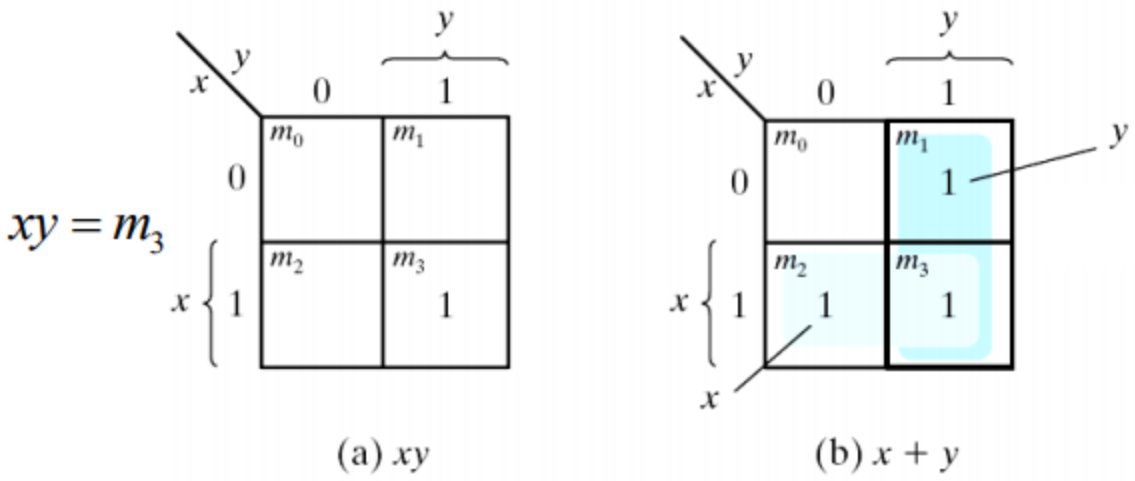
Optimized Implementation of Logic Functions – Karnaugh Map

Two-Variable K-Map

- Four minterms
- x' = row 0; x = row 1
- y' = column 0; y = column 1
- A truth table in square diagram



Two-variable Map



Representation of functions in the map

F (x, y) = Σm (1,2,3)

= x'y + xy' +xy

After minimization

F (x, y) = x + y

Optimized Implementation of Logic Functions – Karnaugh Map

AMRITA VISHWA VIDYAPEETHAM

Three-Variable K-Map

- A three-variable Karnaugh map is constructed by placing 2 two-variable maps side by side.
- Eight minterms
- The Gray code sequence
- Any two adjacent squares in the map differ by only on variable.
- In a three-variable map it is possible to combine cells to produce product terms that correspond to

➢ a single cell – gives 3 variable term

➢ two adjacent cells (pair) – gives 2 variable term

➢ a group of four adjacent cells (quad)- gives single variable term

Optimized Implementation of Logic Functions – Karnaugh Map

Three-Variable K-Map

x_1	x_2	x_3	
0	0	0	m_0
0	0	1	m_1
0	1	0	m_2
0	1	1	m_3
1	0	0	m_4
1	0	1	m_5
1	1	0	m_6
1	1	1	m_7

(a) Truth table

x_3 \ x_1x_2	00	01	11	10
0	m_0	m_2	m_6	m_4
1	m_1	m_3	m_7	m_5

(b) Karnaugh map

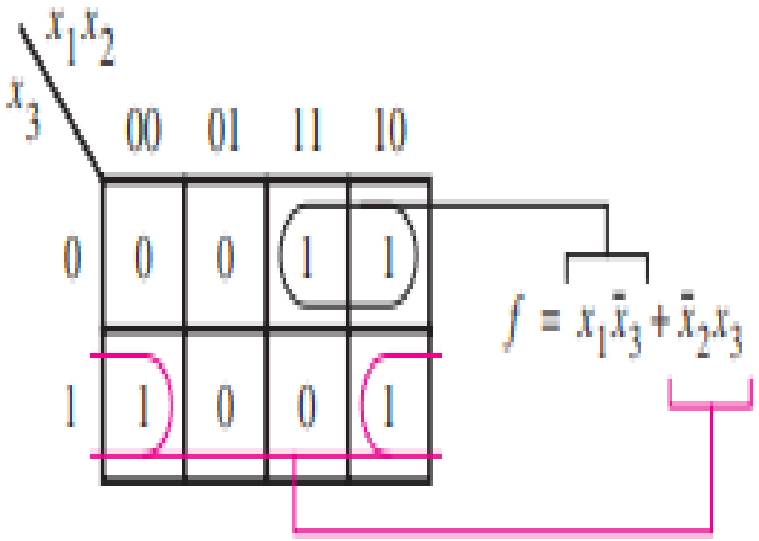
Location of three-variable minterms.

Optimized Implementation of Logic Functions – Karnaugh Map

Three-Variable K-Map

Example 1. Simplify the Boolean function

$$f = \Sigma m(1,4,5,6)$$



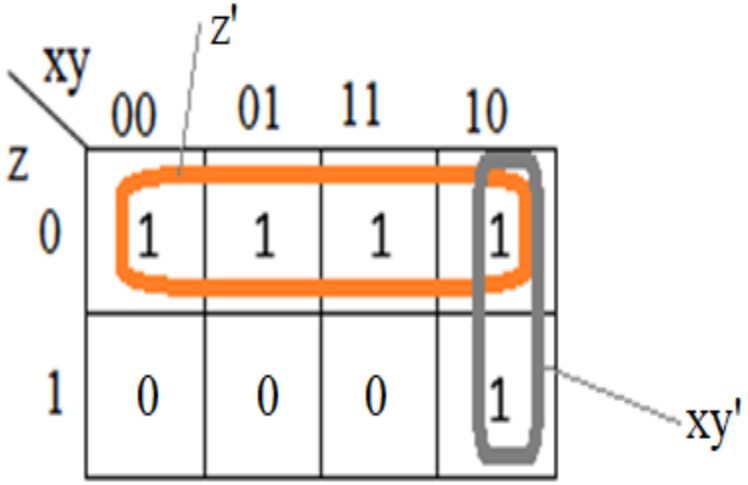
Optimized Implementation of Logic Functions – Karnaugh Map

Three-Variable K-Map

Example**2** : simplify the Boolean functions:

$$F(x, y, z) = \sum(0, 2, 4, 5, 6)$$

$$F(x, y, z) = \sum(0, 2, 4, 5, 6) = z' + xy'$$



Optimized Implementation of Logic Functions – Karnaugh Map

Three-Variable K-Map

Example 3 : Let the Boolean function

$$F = A'C + A'B + AB'C + BC$$

a) Express it in sum of minterms.
b) Find the minimal sum of products expression.

Solution-

a) F = A'C +A'B +AB'C + BC

= A' (B + B') C + A'B (C + C') +AB'C + (A+ A')BC

= A'BC +A'B'C + A'BC + A'BC' +AB'C +ABC + A'BC

$= m_3 + m_1 + m_3 + m_2 + m_5 + m_7 + m_3$

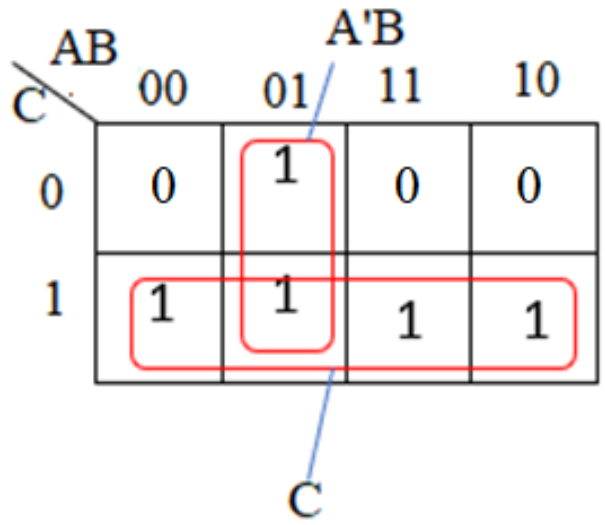
$F = \Sigma\, m(1, 2, 3, 5, 7)$

Optimized Implementation of Logic Functions – Karnaugh Map

Three-Variable K-Map

Solution of Example 3-

b)



$$F(A,B,C)=\sum(1,2,3,5,7)=C+A'B$$

Optimized Implementation of Logic Functions – Karnaugh Map

Three-Variable K-Map

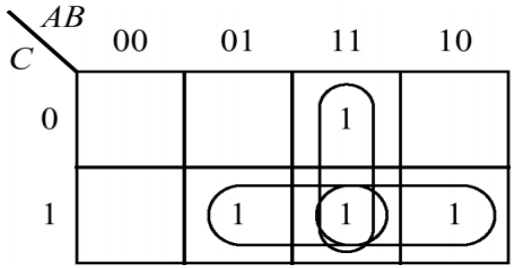
Example 4. Design a 3-bit Majority function.

Solution- **Majority function** is a threshold **function** that produces a 1 if and only if the majority of the inputs are 1.

Truth Table

Minterm Index	A	B	C	F
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1

$F = \Sigma m\,(3,5,6,7)$



$F = BC + AB + AC$

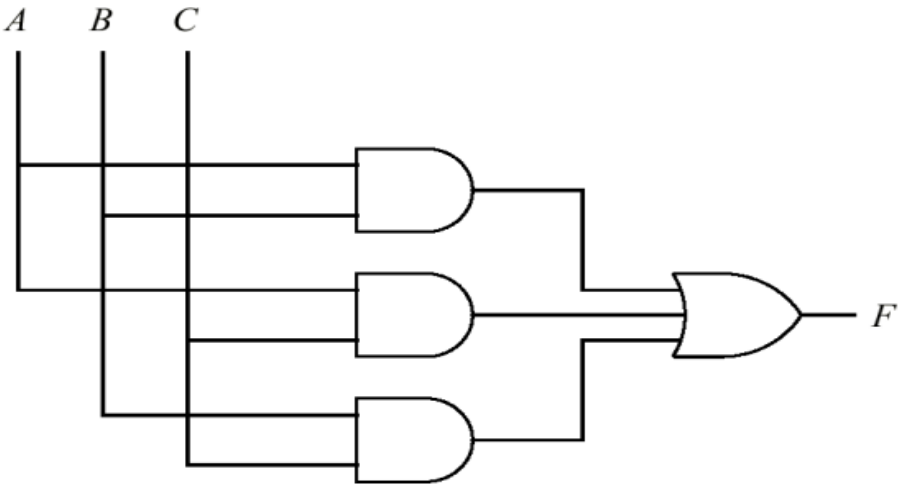
Optimized Implementation of Logic Functions – Karnaugh Map

Three-Variable K-Map

Solution of Example 4-

Logic Circuit for 3-bit Majority function-

$$F = AB + AC + BC$$



LECTURE 8

Optimized Implementation of Logic Functions – Karnaugh Map

AMRITA VISHWA VIDYAPEETHAM

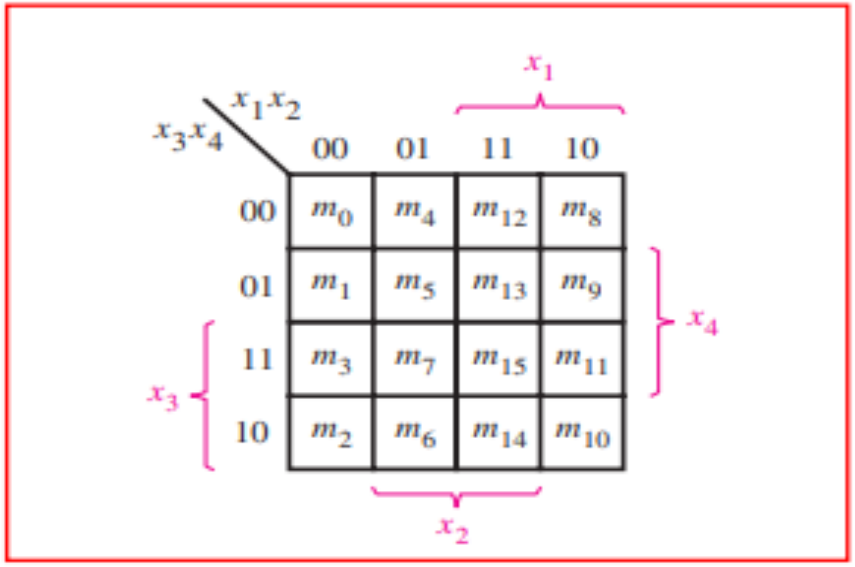
Four-Variable K-Map

- A four-variable map is constructed by placing 2 three-variable maps together to create four rows in the same fashion as we used 2 two-variable maps to form the four columns in a three-variable map.
- In a four-variable map it is possible to combine cells to produce product terms that correspond to a single cell, two adjacent cells (pair), a group of four adjacent cells (quad) and a group of eight adjacent cells(octa).
- Corner cells also can be combined to produce product term that correspond to a pair, quad or octa.

Optimized Implementation of Logic Functions – Karnaugh Map

Four-Variable K-Map

- 16 minterms
- Combinations of 16 adjacent squares



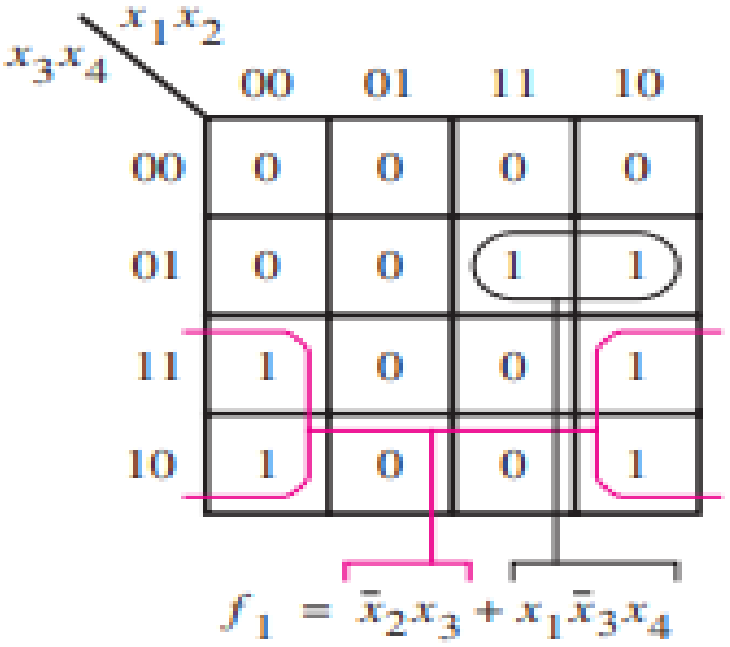
A four-variable Karnaugh map.

Optimized Implementation of Logic Functions – Karnaugh Map

Four-Variable K-Map

Example 1: Simplify the Boolean function-

$f1(x_1,x_2,x_3,x_4) = \Sigma m(2,3,9,10,11,13)$

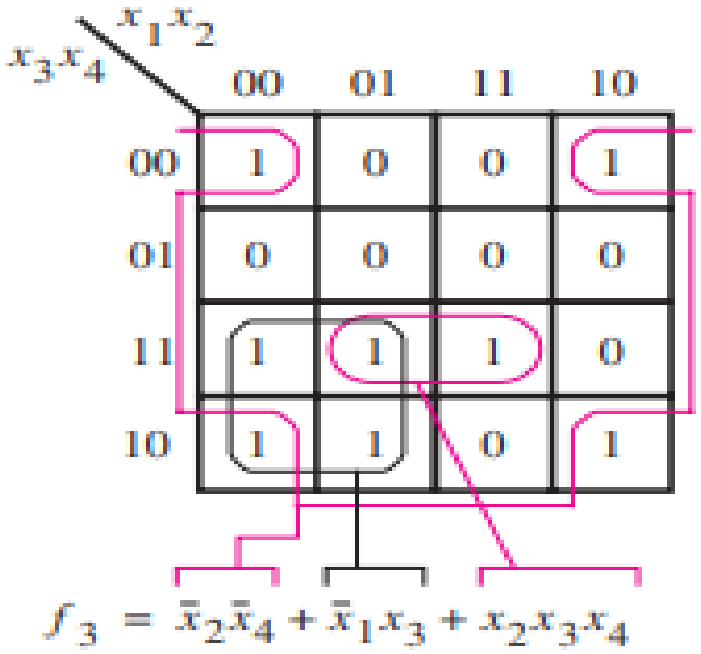


Optimized Implementation of Logic Functions – Karnaugh Map

Four-Variable K-Map

Example 2: Simplify the Boolean function-

$$f1(x_1,x_2,x_3,x_4) = \Sigma m(0,2,3,6,7,8,10,15)$$

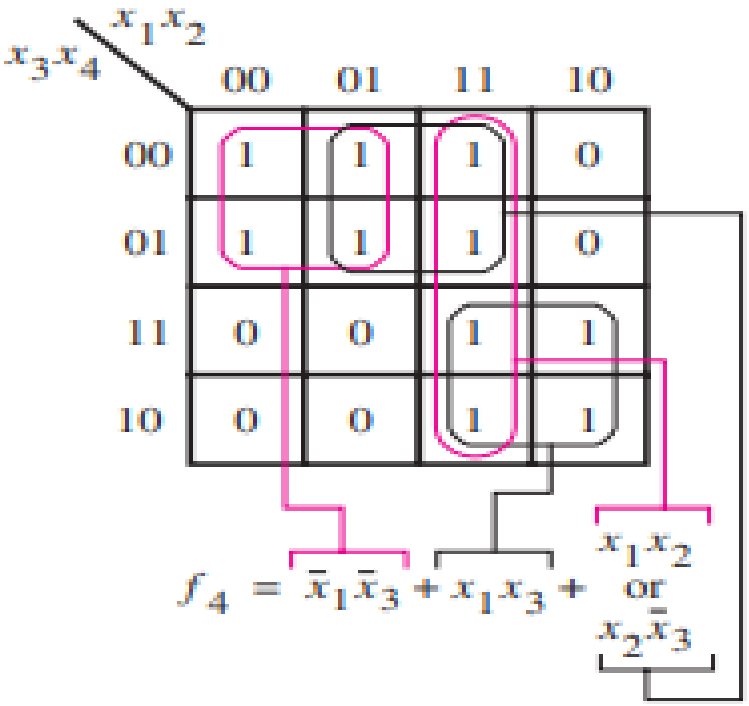


Optimized Implementation of Logic Functions – Karnaugh Map

Four-Variable K-Map

Example 3: Simplify the Boolean function-

$$f1(x_1,x_2,x_3,x_4) = \Sigma m(0,1,4,5,10,11,12,13,14,15)$$



Optimized Implementation of Logic Functions – Karnaugh Map

Four-Variable K-Map

Example 4. Simplify the given Boolean function using Karnaugh Map.

$$F(w, x, y, z) = w'x'y'z + x'yz + wy'z' + wxy'z + wx'y'z$$

Solution – First we need to express the function in sum of minterms form:

$$F = w'x'y'z + (w + w')x'yz + w(x + x')\,y'z' + wxy'z + wx'y'z$$

$$= w'x'y'z + wx'yz + w'x'yz + wxy'z' + wx'y'z' + wxy'z + wx'y'z$$

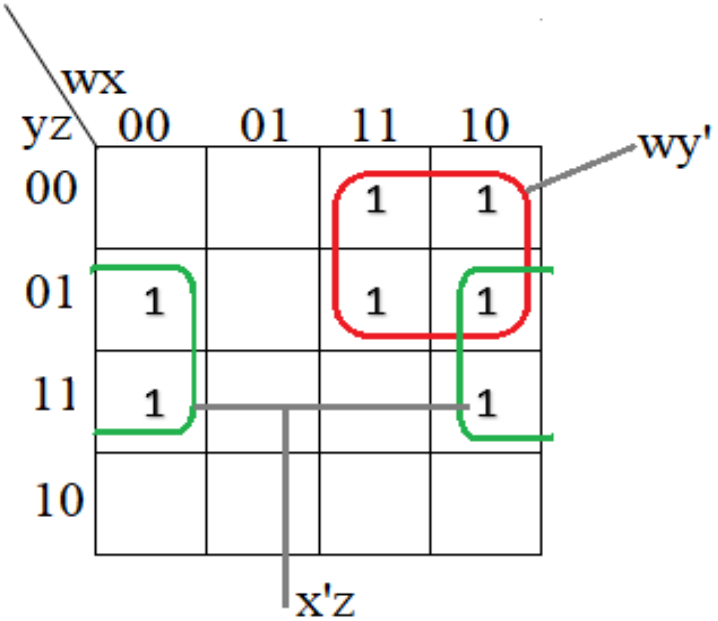
$$= m_1 + m_{11} + m_3 + m_{12} + m_8 + m_{13} + m_9$$

$$F = \Sigma\, m(1,3,8,9,11,12,13)$$

Optimized Implementation of Logic Functions – Karnaugh Map

Four-Variable K-Map

Solution for Example 4-



$F = wy' + x'z$

Optimized Implementation of Logic Functions – Karnaugh Map

Five-Variable K-Map

- We can use 2 four-variable maps to construct a five-variable map.
- It is easy to imagine a structure where one map is directly behind the other, and they are distinguished by A = 0 for one map and A = 1 for the other map.

DE \ BC	00	01	11	10
00	m0	m4	m12	m8
01	m1	m5	m13	m9
11	m3	m7	m15	m11
10	m2	m6	m14	m10

A = 0

DE \ BC	00	01	11	10
00	m16	m20	m28	m24
01	m17	m21	m29	m25
11	m19	m23	m31	m27
10	m18	m22	m30	m26

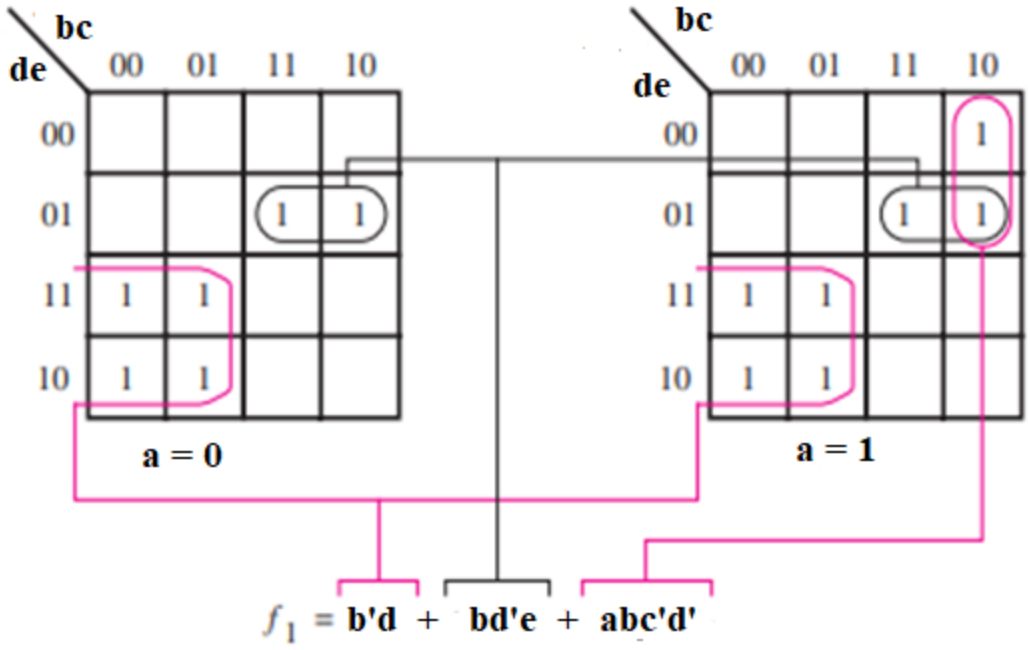
A = 1

Optimized Implementation of Logic Functions – Karnaugh Map

Five-Variable K-Map

Example 1. Simplify the Boolean function

$$f_1(a, b, c, d, e) = \Sigma\, m(2,3,6,7,9,13,18,19,22,23,24,25,29)$$

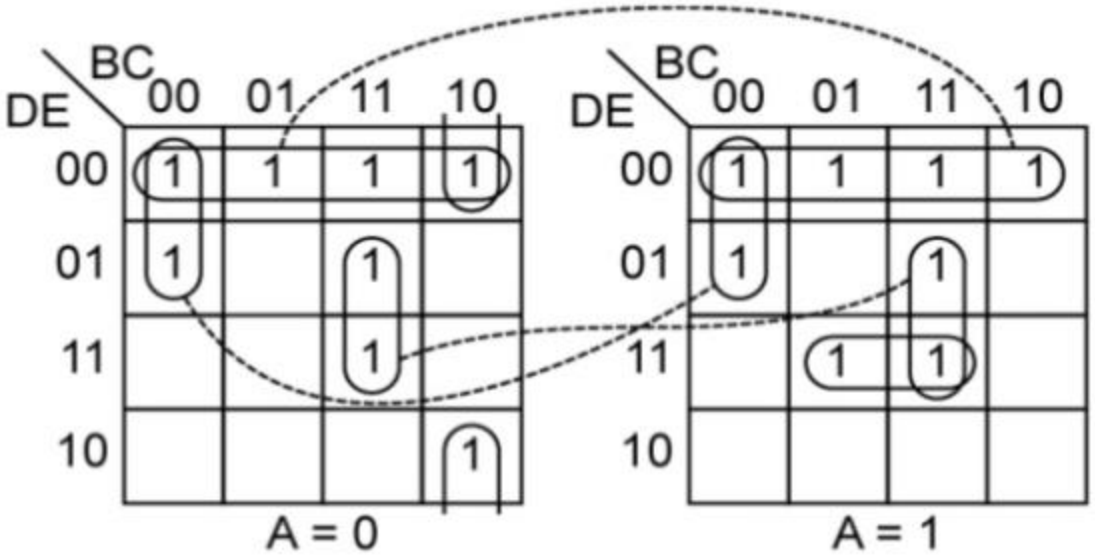


Optimized Implementation of Logic Functions – Karnaugh Map

Five-Variable K-Map

Example 2. Simplify the Boolean function

$$F(A,B,C,D,E) = \sum m(0, 1, 4, 8, 10, 12, 13, 15, 16, 17, 20, 23, 24, 28, 29, 31)$$



$$F = D'E' + B'C'D' + BCE + A'BC'E' + ACDE$$

Optimized Implementation of Logic Functions – Karnaugh Map

of Adjacent Squares and # of Literals

- The relationship between the number od adjacent squares and the number of literals in term-

	Number of Adjacent Squares	Number of Literals in a Term in an *n*-variable Map			
K	2^k	***n* = 2**	***n* = 3**	***n* = 4**	***n* = 5**
0	1	2	3	4	5
1	2	1	2	3	4
2	4	0	1	2	3
3	8		0	1	2
4	16			0	1
5	32				0